PAPER TRADE

Project PRD, Technical Context & Development Handoff

A detailed single-source-of-truth document covering the product vision, architecture, development environment, completed setup, database/backend configuration, CleverTap objectives, development practices, and the next implementation phase.

Document	Paper Trade — Project Context & PRD
Project type	Educational/demo paper-trading Android application
Primary stack	Android/Kotlin/Compose + Java/Spring Boot + PostgreSQL
Local database	PostgreSQL 18 via Docker
Analytics / SDK goal	CleverTap Android SDK integration
Current phase	Infrastructure complete; frontend development next

Handoff status: The backend is working, PostgreSQL is connected, the Android project is created, and the next chat can begin frontend development without repeating environment setup.

1. Product Vision

Paper Trade is a small Groww-inspired paper-trading application built primarily as a learning and SDK-integration project.

The application should feel like a small, realistic product rather than a collection of disconnected demo screens. It intentionally uses a frontend, backend, and persistent database so that user actions produce real state changes.

The project is **not** intended to become a production-grade brokerage or financial platform. The goal is a compact application with enough realistic behavior to learn Android development, backend APIs, database persistence, and CleverTap event-driven use cases.

Core product goal

Build a simple Android paper-trading experience in which a user can sign up, receive a $1,000 virtual balance, view stock prices, buy and sell stocks using virtual money, and see the resulting portfolio and performance. The application should simultaneously generate useful CleverTap events for experimentation.

One-sentence definition

Build a small, realistic Groww-inspired paper-trading Android app with Kotlin/Compose, a Java/Spring Boot backend, PostgreSQL persistence, and CleverTap SDK integration, while keeping the implementation simple enough to understand and use as a learning/demo project.

2. Product Principles & Practices

Keep the scope deliberately small

- No real-money payments or financial transactions.
- No options/futures, advanced order books, complex trading algorithms, or advanced technical analysis.
- No unnecessary social/chat features or enterprise-level infrastructure.
- Only add libraries, services, or architecture patterns when they have a clear learning or product reason.

Learn while building

The user wants to understand what the code is doing, not blindly copy a large application. New concepts should therefore be introduced with small, practical examples. Spring Boot annotations such as **@RestController**, **@GetMapping**, **@PostMapping**, **@Entity**, **@Id**, **@Service**, and **@Repository** should be explained in simple language when they are introduced.

Development style

- Build one piece at a time and verify it before moving on.
- Prefer concise explanations and practical commands.
- Avoid dumping the entire application at once.
- Do not restart setup work that has already been completed.
- Keep the implementation beginner-friendly and understandable.

3. Intended User Journey

```
Open App
↓
Demo Animation
↓
Login / Signup
↓
Create Account
↓
Receive $1,000 virtual balance
↓
Stocks Screen
↓
View Stock
↓
Buy / Sell Stock
↓
Backend validates and processes trade
↓
Database updates balance + holdings + trade
↓
CleverTap records relevant event
↓
Dashboard
↓
View holdings + P/L + available cash
```

Definition of Done for the first meaningful version

- User can open the Android app and see the demo introduction.
- User can sign up or log in.
- A new user receives $1,000 virtual money.
- User can see stock information from a free market-data API.
- User can open a stock.
- User can buy a stock and have virtual balance decrease.
- The purchased stock appears in holdings.
- User can sell a stock and have the balance update.
- Dashboard shows holdings, available money, and basic profit/loss.
- Important state persists in PostgreSQL.
- Meaningful CleverTap events are generated throughout the journey.

4. Main Application Screens

4.1 Demo / Intro

A subtle opening animation displays the message: **“This is a Demo project for paper trading”**. The animation should remain simple and should not become a major feature.

4.2 Login / Signup

Users can create an account or log in. The exact authentication mechanism can remain simple because this is a demo project.

4.3 Stocks

The main screen displays different stocks, ideally using a free market-data API. Each stock should show the company name, symbol, current/recent price, company icon/logo, and net positive/negative change.

4.4 Trade

The user can buy or sell a selected stock with a specified quantity. The backend validates the operation and persists the resulting state.

4.5 Dashboard

The dashboard shows available money, invested amount, portfolio value, profit/loss, and a simple list of holdings.

Intended primary navigation: Stocks | Trade | Dashboard

5. CleverTap Objectives

CleverTap is a major reason for building this application. The product should contain enough meaningful user actions to demonstrate event tracking, user profiles, segmentation, campaigns, conversion goals, control-group concepts, push notifications, and other relevant SDK capabilities.

Initial event set

Area	Event
Authentication	User Signed Up
Authentication	User Logged In
Onboarding	Onboarding Completed
Discovery	Stock Viewed
Discovery	Stock Search (optional)
Trading	Buy Clicked
Trading	Sell Clicked
Trading	Trade Completed
Trading	Trade Failed
Portfolio	Dashboard Viewed
Portfolio	Portfolio Viewed
Portfolio	Holding Viewed
Portfolio	Profit/Loss Viewed

Useful event properties

```
symbol
company_name
price
quantity
trade_value
trade_type
```

Example trade event

```
Trade Completed
symbol = AAPL
price = 200
quantity = 2
trade_type = BUY
trade_value = 400
```

Potential CleverTap segments

- Users who viewed NVIDIA but never bought NVIDIA.
- Users who completed 3 or more trades.
- Users who signed up but never traded.
- Users whose portfolio is profitable.

6. Overall Technical Architecture

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Android App ■
■ Kotlin + Jetpack Compose ■
■ FRONTEND ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■
■ HTTP / REST API
▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Spring Boot ■
■ Java Backend ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■
■ JPA / SQL
▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ PostgreSQL ■
■ DATABASE ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

Android App ■■■■■■■■■■■■■■■■→ CleverTap
(user/event tracking)
```

This is a standard three-layer application shape for the scale of this project: the Android app handles presentation and user interaction, Spring Boot handles API/application logic, and PostgreSQL persists the application state.

7. Why We Chose Frontend + Backend + Database

We explicitly discussed whether the project could be built without a database. The conclusion was that a database is appropriate because the app should behave like a small real application and preserve state.

Layer	Technology	Purpose
Frontend	Android + Kotlin + Jetpack Compose	UI and user interaction
Backend	Java + Spring Boot	API and application/business logic
Database	PostgreSQL	Persistent user/trading state
ORM	Spring Data JPA / Hibernate	Object/database persistence
Local DB environment	Docker	Repeatable local PostgreSQL setup

Layer	Technology	Purpose
Analytics / SDK	CleverTap	Events, profiles, segmentation, campaigns
Source control	Git + GitHub	Version control and collaboration

8. Project Directory & Repository Structure

```
~/Programming/
Paper_Trade/
Paper_Trade_frontend/ ← Android / Kotlin / Jetpack Compose
Paper_Trade_backend/ ← Java / Spring Boot
```

The workspace was intentionally cleaned before creating the project, so the current project structure is the chosen starting point rather than a legacy codebase.

9. Android Frontend — Completed Setup

Setting	Current value
Project	Paper_Trade_frontend
Language	Kotlin
UI	Jetpack Compose
Template	Empty Activity
Package / Application ID	com.fredrick.papertrade
Minimum SDK	24
Compile SDK	37
Target SDK	37
Compose plugin	kotlin.compose
Compose enabled	buildFeatures.compose = true

Compose is already configured

The generated Android project already contains the Compose BOM and core Compose dependencies, including Activity Compose, Material 3, UI, UI Graphics, UI Tooling Preview, Core KTX, and Lifecycle Runtime KTX. No additional Compose setup is required before starting UI development.

Android Studio's Compose Preview is already visible. This is the UI preview and is distinct from the Android Emulator.

10. Backend — Completed Setup

Setting	Current value
Project	Paper_Trade_backend
Project type	Gradle - Kotlin
Language	Java
Spring Boot	4.1.1
Group	com.fredrick
Artifact	paper-trade-backend
Name	Paper Trade Backend
Package	com.fredrick.papertrade
Packaging	Jar
Java	17
Dependencies	Spring Web; Spring Data JPA; PostgreSQL Driver; Validation; DevTools

Backend health endpoint

```
// src/main/java/com/fredrick/papertrade/HealthController.java

package com.fredrick.papertrade;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HealthController {

@GetMapping("/")
public String serverStatus() {
return "Server is ON";
}
}
```

@RestController tells Spring that the class contains HTTP endpoints and returned values should be sent as response bodies. **@GetMapping("/")** maps an HTTP GET request to the root path to the Java method.

The endpoint is intentionally tiny and is currently used as a health check confirming that Spring Boot is running.

11. Backend Runtime Status

Spring Boot runs on port **8080**. On the Mac, the working health check is:

```
http://localhost:8080
```

The response is:

```
Server is ON
```

Before the datasource was configured, Spring Boot failed with a DataSource configuration error because Spring Data JPA and the PostgreSQL driver were present but no database URL/configuration was supplied. This was a configuration issue, not an installation problem. After PostgreSQL and datasource settings were configured, Spring Boot started successfully.

12. PostgreSQL + Docker — Completed Setup

Setting	Current value
Container	paper-trade-db
Image	postgres:18
Database	paper_trade
Username	papertrade
Password (local demo)	papertrade
Host port	5432
Persistent volume	paper-trade-data
Container data mount	/var/lib/postgresql

Working Docker command

```
docker run --name paper-trade-db \
-e POSTGRES_DB=paper_trade \
-e POSTGRES_USER=papertrade \
-e POSTGRES_PASSWORD=papertrade \
-p 5432:5432 \
-v paper-trade-data:/var/lib/postgresql \
-d postgres:18
```

Why the volume path matters

The first attempt mounted PostgreSQL 18 data at /var/lib/postgresql/data. The PostgreSQL 18 Docker image reported that this layout was not appropriate for its current cluster-directory structure and recommended mounting /var/lib/postgresql. The container was removed and recreated using the correct mount.

Database verification

```
docker exec -it paper-trade-db psql -U papertrade -d paper_trade
```

This connection was successfully tested, confirming that PostgreSQL itself is working.

13. Spring Boot → PostgreSQL Configuration

```
spring.datasource.url=jdbc:postgresql://localhost:5432/paper_trade
spring.datasource.username=papertrade
spring.datasource.password=papertrade

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

This configuration successfully connected the Spring Boot application to the PostgreSQL Docker container.

Important: These credentials are local-demo credentials only. Before pushing the project to GitHub, credentials should be moved to environment variables or another local-only configuration mechanism. Real secrets must never be committed to the repository.

14. Android Emulator Networking

The emulator is a separate virtual device. Therefore, **localhost inside the emulator is not the same as localhost on the Mac**.

Where	URL	Meaning
Mac browser	http://localhost:8080	Spring Boot running on the Mac
Android emulator	http://localhost:8080	Emulator's own localhost — not the Mac
Android emulator	http://10.0.2.2:8080	Special emulator address that reaches the Mac's localhost

```
Android Emulator
↓
10.0.2.2:8080
↓
Mac
↓
Spring Boot
↓
Server is ON
```

Therefore, the emulator can already communicate with the backend. What has not yet been implemented is Android Kotlin code that sends API requests. The 10.0.2.2 address is a local development/emulator networking detail, not the production backend URL.

15. Development Environment

Tool / Environment	Current setup
Machine	Apple Silicon MacBook Air
Architecture	arm64
Apple Clang	21.0.0
Git	2.33.0
Xcode Command Line Tools	Installed
Java available via company Self Service	Zulu JDK 18.0.1
Backend Java target	17
Android Studio Gradle JVM	Version 17, Any vendor
Python	3.14.8
pip	26.2.1
nvm	0.40.3
Node	24.21.0
npm	11.19.0
IDEs	Android Studio, IntelliJ IDEA, VS Code
Android SDK	~/Library/Android/sdk
Android API	37 (CinnamonBun)
Docker	29.8.2

The Mac is company-managed and the user does not have sudo privileges. Docker Desktop was therefore installed through the company's Self Service rather than relying on system-level installation.

16. GitHub & Collaboration

GitHub is already connected/configured in VS Code. The intention is to push the project to GitHub after the initial setup is ready.

Before the first push

- Do not commit PostgreSQL passwords or real secrets.
- Add/verify appropriate .gitignore rules.
- Prefer environment variables for database credentials.
- Consider committing a Docker Compose configuration so collaborators can recreate the local PostgreSQL environment.

A future collaboration flow can be:

```
git clone <repository>
↓
docker compose up -d
↓
PostgreSQL starts
↓
Run Spring Boot
↓
Run Android app
```

17. Current Project Status

Status	Item
DONE	macOS development environment configured
DONE	Android Studio / SDK / emulator configured
DONE	Kotlin + Jetpack Compose frontend created
DONE	Spring Boot Java backend created
DONE	PostgreSQL selected and Docker Desktop installed
DONE	PostgreSQL Docker container created
DONE	Persistent Docker volume created
DONE	PostgreSQL connection verified
DONE	Spring Boot → PostgreSQL connection verified
DONE	Spring Boot starts successfully
DONE	Health endpoint created
DONE	GET / returns Server is ON
READY	Emulator can use 10.0.2.2:8080 to reach Mac backend
TODO	Android UI development
TODO	Android → Spring Boot API communication
TODO	Database entities/models
TODO	Signup/login implementation
TODO	Virtual balance implementation
TODO	Stock API integration
TODO	Stock listing and details UI
TODO	Buy/sell functionality
TODO	Portfolio/dashboard
TODO	CleverTap SDK integration
TODO	CleverTap event implementation/testing
TODO	Security/configuration cleanup
TODO	Final GitHub push/setup

18. Recommended Development Sequence

```
1. Demo animation
↓
2. Login / Signup
↓
3. Onboarding + $1,000 balance
↓
4. Main navigation
↓
5. Stocks screen
↓
6. Stock details
↓
7. Buy / Sell screen
↓
8. Dashboard
↓
9. Android → Spring Boot API communication
↓
10. Database-backed functionality
↓
11. CleverTap SDK integration
↓
12. CleverTap events / segments / campaigns
```

The exact ordering can be adjusted while building, but the principle is to implement and verify one small piece at a time.

19. Backend Data Model — Intended Direction

The exact entity design has not yet been implemented. The following represents the intended product state, not completed code.

Concept	Purpose
User	Account identity and user information
Balance / Wallet	Virtual cash available to the user
Holding	Current quantity of a stock owned by the user
Trade	Record of a buy or sell transaction
Stock / Market data	Current or recent stock information as needed by the app
Portfolio	Calculated/aggregated view of holdings and performance

Do not treat this table as a finalized database schema. The schema should be designed as implementation begins and only as much as the product actually needs.

20. API Direction — Intended, Not Yet Implemented

The backend health endpoint is the only endpoint currently created. Future endpoints should be introduced incrementally.

Possible API	Purpose
GET /	Health check — already implemented
POST /auth/signup	Create user account

Possible API	Purpose
POST /auth/login	Authenticate user
GET /stocks	Return stock data
GET /stocks/{symbol}	Return selected stock
POST /trades	Create a buy/sell trade
GET /holdings	Return user's holdings
GET /portfolio	Return portfolio summary

These endpoint names are planning examples, not committed contracts. Final names and request/response structures should be chosen during implementation.

21. Market Data Direction

The stock screen should ideally use a free stock/market-data API. The exact provider has not yet been selected. The selection should prioritize a free tier, reasonable simplicity, required stock fields, and suitability for a demo/learning application.

The app needs only enough market data to show stock symbol, company name, price, change, and optionally an icon/logo. The project does not require sophisticated real-time trading infrastructure.

22. Important Technical Understanding Established

- The Android emulator is separate from the Mac and has its own localhost.
- 10.0.2.2 is the standard Android emulator route to the host machine's localhost.
- The frontend does not need to be connected to the backend before the backend can be independently tested.
- The backend and database are already connected; frontend API calls are the next integration step.
- The 404 that originally appeared at localhost:8080 was not a server failure; it meant there was no controller mapped to the root path yet.
- After HealthController was added, GET / returns Server is ON.
- Docker's PostgreSQL volume makes database state persistent across container stop/restart operations.

23. Security & Repository Hygiene

The current setup is intended for local development. Before the project is treated as shareable or pushed publicly, configuration hygiene must be improved.

- Move DB username/password into environment variables or local-only configuration.
- Do not commit real credentials.
- Review .gitignore before the first push.
- Do not place CleverTap credentials or other production secrets directly in source code if they should remain private.
- Use development-specific URLs such as 10.0.2.2 only for local emulator testing; production builds should use the deployed backend URL.

No production security architecture has been designed yet, and that is acceptable for the current demo phase.

24. Handoff Instructions for the Next Development Chat

Do not redo environment setup. Start from the current project state described here.

The next chat should assume the following:

```
Project root:
~/Programming/Paper_Trade/

Frontend:
~/Programming/Paper_Trade/Paper_Trade_frontend

Backend:
~/Programming/Paper_Trade/Paper_Trade_backend

Backend:
Spring Boot 4.1.1 / Java 17 / port 8080

Database:
PostgreSQL 18 / Docker / port 5432

Health check:
GET http://localhost:8080/
→ Server is ON

Android emulator → Mac backend:
http://10.0.2.2:8080
```

The immediate focus should be **Android frontend development**. Build the UI progressively, explain the relevant Compose concepts, and avoid rebuilding or reconfiguring the already-working infrastructure unless an actual problem requires it.

25. Final Project Snapshot

Dimension	Decision / State
Product	Small Groww-inspired paper-trading demo
Primary learning goals	Android, Kotlin, Compose, APIs, Spring Boot, Java, PostgreSQL, Docker, CleverTap
Frontend	Android + Kotlin + Jetpack Compose
Backend	Java + Spring Boot 4.1.1
Database	PostgreSQL 18
Database environment	Docker Desktop
Analytics / SDK	CleverTap Android SDK
Initial balance	$1,000 virtual money
Primary screens	Stocks, Trade, Dashboard
Authentication	Login / Signup
Market data	Free API to be selected
Backend health endpoint	GET / → Server is ON
Development host	Mac / localhost:8080
Emulator host route	10.0.2.2:8080
Persistence	PostgreSQL via JPA/Hibernate
Source control	Git + GitHub
Current phase	Setup complete; frontend development next

End of handoff. This document is intended to be the detailed source of truth for continuing the Paper Trade project in a new chat.